TRIBHUVAN UNIVERSITY

Faculty of Humanities and Social Sciences

Pashupati Multiple Campus

A Project Report on

HAMRO PRACHIN SAMPADA: A 3D HERITAGE EXPLORER AND QUIZ GAME FOR NEPAL

Submitted to

Department of Computer Application

Pashupati Multiple Campus

In partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications (BCA)

Project III (CACS452), Eighth Semester

Submitted by

Niranjan Katwal

TU Symbol No.: 27802063

TU Registration No.: 6-2-278-18-2022

Project Supervisor

Name and designation of the supervisor

Month and year of submission

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project report, prepared under my supervision by **Niranjan Katwal** (TU Symbol No. 27802063, Registration No. 6-2-278-18-2022) and entitled "**Hamro Prachin Sampada: A 3D Heritage Explorer and Quiz Game for Nepal**", be accepted as partial fulfillment of the requirements for the degree of Bachelor of Computer Applications. In my opinion, this report is satisfactory for submission to the evaluation committee.

Signature

Name of the supervisor

Designation

Pashupati Multiple Campus

Date: dd/mm/yyyy

CERTIFICATE OF APPROVAL

This is to certify that the project report entitled "**Hamro Prachin Sampada: A 3D Heritage Explorer and Quiz Game for Nepal**" submitted by **Niranjan Katwal** in partial fulfillment of the requirements for the degree of Bachelor of Computer Applications has been examined and approved by the evaluation committee.

Signature Supervisor: Name

Signature Internal Examiner: Name

Signature External Examiner: Name

Signature Head of Department / Program Coordinator: Name

Signature Campus Chief: Name

Date: dd/mm/yyyy

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, name of the supervisor, for the guidance, patient review and constructive suggestions that shaped this project. I am thankful to the Head of Department and all the teachers of the Department of Computer Application at Pashupati Multiple Campus for the knowledge they shared during the four years of the BCA program.

I thank Tribhuvan University for the project framework that gave this work a clear direction. I am grateful to the Wikimedia Commons contributors, the OpenStreetMap and OpenFreeMap communities and the Poly Haven artists, whose freely licensed photographs, maps and textures made a heritage project possible for a student. I also thank my classmates and friends who tested early versions and gave honest feedback, and my family for their constant support.

Any remaining mistakes are my own, and I welcome suggestions for improvement.

Niranjan Katwal

ABSTRACT

Nepal holds a large and well documented heritage, yet most young people meet it only through textbooks and scattered photographs. Existing websites show a few famous sites, rarely let the visitor walk through one, and do not turn learning into something to play. This project presents **Hamro Prachin Sampada** ("Our Ancient Heritage"), a free bilingual web application that lets a visitor explore 116 heritage sites across all seven provinces of Nepal.

The client is built with React and Vite, and the server with Express, Socket.io and MySQL. Every site has a story in English and Nepali, photographs from Wikimedia Commons, a map position, and its own hand-built low-poly 3D model that runs in the browser with three.js and has day and dusk lighting. A yatra planner chooses a walking or taxi route through chosen sites within a time budget using greedy insertion and 2-opt improvement. An adaptive quiz of 1,160 questions adjusts difficulty with an Elo rating and schedules missed questions with SM-2 spaced repetition. Find the Monument scores a guess on a map with the Haversine distance, and live multiplayer hunts run in Socket.io rooms with server-side timing. A recommender, a heritage passport with GPS stamps and badges, a leaderboard, and email notifications complete the product.

The system was checked with 12 unit tests, 18 API flow tests, a multiplayer hunt simulation and a browser flow suite that drives the real pages on desktop and mobile widths. The 3D models are approximations drawn from photographs and fact sheets, and the Nepali text and quiz content were drafted with AI assistance and still need review by a native speaker and a heritage expert before public release.

Keywords: digital heritage, Nepal, three.js, WebGL, gamification, orienteering problem, Elo rating, spaced repetition, Socket.io, React, MySQL

TABLE OF CONTENTS

SUPERVISOR'S RECOMMENDATION i
CERTIFICATE OF APPROVAL ii
ACKNOWLEDGEMENT iii
ABSTRACT iv
TABLE OF CONTENTS v
LIST OF ABBREVIATIONS vii
LIST OF FIGURES viii
LIST OF TABLES ix
CHAPTER 1: INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Limitations 2
1.5 Significance 3
1.6 Development Methodology 3
1.7 Report Organization 4
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW 5
2.1 Background Theory 5
2.2 Review of Existing Systems 6
2.3 Review of Related Research 8
2.4 Research Gap 8
CHAPTER 3: SYSTEM ANALYSIS 9
3.1 Requirement Identification 9
3.2 Feasibility Study 10
3.3 Use Case Model 11
3.4 Data Modeling 13
3.5 Process Modeling 15
3.6 Data Collection and Verification 16
CHAPTER 4: SYSTEM DESIGN 19
4.1 Architecture Design 19
4.2 Database Design 20
4.3 Algorithm Design 21
4.4 3D Model Design 27
4.5 User Interface Design 29
4.6 API Design 29
4.7 Security Design 30

CHAPTER 5: IMPLEMENTATION AND TESTING 31
5.1 Tools and Technologies 31
5.2 Module Implementation 31
5.3 Screens 32
5.4 Testing 33
5.5 Results and Discussion 36
5.6 Limitations Found 37
CHAPTER 6: CONCLUSION AND FUTURE WORK 39
6.1 Conclusion 39
6.2 Lessons Learned 39
6.3 Future Work 40
REFERENCES 41
APPENDIX A: RUNNING THE SYSTEM 44

LIST OF ABBREVIATIONS

2D / 3D	Two / Three Dimensional
API	Application Programming Interface
BCA	Bachelor of Computer Applications
CC	Creative Commons
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ER	Entity Relationship
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
OSM	OpenStreetMap
REST	Representational State Transfer
SM-2	SuperMemo 2 spaced repetition algorithm
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TU	Tribhuvan University
UI	User Interface
UML	Unified Modeling Language
UNESCO	United Nations Educational, Scientific and Cultural Organization
WebGL	Web Graphics Library

LIST OF FIGURES

Figure 3.1: Project schedule (Gantt chart) 11
Figure 3.2: Use case diagram of Hamro Prachin Sampada 12
Figure 3.3: Entity relationship diagram 14
Figure 3.4: DFD level 1 16
Figure 4.1: System architecture of Hamro Prachin Sampada 19
Figure 4.2: Entity relationship diagram 20
Figure 4.3: Flow of the yatra planner 22
Figure 4.4: Flow of the adaptive quiz and spaced repetition 24
Figure 4.5: Sequence of one Find the Monument game 26
Figure 4.6: Pipeline from research notes to a 3D scene 28
Figure 5.1: Home page with the hero model and the seven provinces 32
Figure 5.2: Explore: the Nepal map with province filters and the site list 32
Figure 5.3: 3D model of a site in the dusk-aarti setting 33
Figure 5.4: Yatra planner with the timed plan and its walk and taxi legs 33
Figure 5.5: Live hunt with the room code and the leaderboard 33

LIST OF TABLES

Table 1.1: Iterations of the development 4
Table 2.1: Comparison of Sampada with existing systems 7
Table 3.1: Functional requirements 9
Table 3.2: Non-functional requirements 10
Table 3.3: Estimated cost of development and a first year of small deployment 10
Table 3.4: Use case description: Plan a heritage walk 12
Table 3.5: Use case description: Answer adaptive quiz questions 12
Table 3.6: Entities and their main attributes 14
Table 3.7: Sites in the dataset by province 16
Table 4.1: Tables of the database 21
Table 4.2: REST endpoints by router 30
Table 4.3: Threats and measures 30
Table 5.1: Tools and libraries (exact versions from package.json) 31
Table 5.2: Unit test cases 34
Table 5.3: API test cases 34
Table 5.4: Planner running time (measured, 20 runs) 36
Table 5.5: API response time (60 sequential requests) 36
Table 5.6: Files fetched on the first visit (production build) 37

CHAPTER 1: INTRODUCTION

1.1 Background

Nepal has one of the densest concentrations of living heritage in South Asia. The Kathmandu Valley alone is a UNESCO World Heritage property made of seven monument zones: the Durbar Squares of Kathmandu, Patan and Bhaktapur, the stupas of Swayambhunath and Boudhanath, Pashupatinath and Changu Narayan [1]. Lumbini, the birthplace of the Buddha, is a second Nepali cultural property [2]. Beyond these lie hundreds of smaller shrines, courtyards, forts, ponds and pilgrimage sites in all seven provinces, many known only locally. The 2015 Gorkha earthquake damaged many of them, and restoration began only in the following years [3], which makes careful documentation more valuable.

Today a visitor with a smartphone can look up most of this information, but it is spread over encyclopedia pages, news articles, travel blogs and photo archives of uneven depth, mostly in English, and none of it helps with the practical questions of a visit: which sites are open, how far apart they are, and what fits into three hours. Modern browsers can also draw 3D scenes with WebGL, read the device location, speak text aloud and keep a live connection with a server, so a heritage guide can be interactive and playful.

Hamro Prachin Sampada (हाम्रो प्राचीन सम्पदा, "our ancient heritage") is a web application built around this opportunity. It collects 116 heritage sites across all seven provinces into one dataset and turns it into a story map, a 3D model of every site, a heritage walk planner, an adaptive quiz, a photo-based guessing game, live multiplayer hunts and a passport that records visited places. The interface is available in English and Nepali.

1.2 Problem Statement

The project addresses the following problems.

- **Scattered information.** Facts about a monument are spread over many sites of different quality; no single place shows history, legend, festivals, visiting hours, fees and sources together.
- **Neglected smaller sites.** Small temples in the Gandaki, Karnali, Sudurpashchim, Koshi, Madhesh and Lumbini provinces have little organized, bilingual content.
- **Passive presentation.** Most heritage websites are read-only and do nothing to make a visitor look closely, remember or explore.

- **No visit planning.** Opening hours, distances and time at each site decide what a day can include, but visitors work this out by hand.
- **Limited 3D understanding.** High-quality 3D scans exist for few Nepali monuments and are often paid or hosted on separate platforms.
- **Language barrier.** Bilingual heritage material is rare, and most game-style learning tools do not cover Nepal.

The problem is therefore how to bring verified heritage information, planning support and engaging learning activities for Nepal into one free web application that works on an ordinary phone or laptop.

1.3 Objectives

The general objective is to develop a free, bilingual web application that helps people discover, understand and visit the heritage of Nepal. The specific objectives are:

1. To build a dataset of 116 heritage sites in all seven provinces, with coordinates, history, festivals, visiting information, licensed photographs and sources.
2. To generate an interactive 3D model of every site from a structured description.
3. To plan a one-day heritage walk that respects opening hours and the time available.
4. To provide an adaptive quiz, a location game and a live multiplayer hunt with server-side scoring.
5. To keep a passport with GPS-verified stamps and badges, and an administrator console for managing content.
6. To test the main algorithms and the API with automated tests.

1.4 Scope and Limitations

1.4.1 Scope

The system is a web application with a React client and an Express server backed by MySQL. It covers browsing and searching the dataset, the map, the 3D viewer, the audio guide, the English and Nepali interface, recommendations, the walk planner, the quiz, the Find the Monument game, live hunts, the passport and badges, leaderboards, accounts with email verification, and an administrator console. Two roles exist: explorer (registered visitor) and administrator. Guests can use almost every feature without an account.

1.4.2 Limitations

- The dataset of 116 sites is a curated selection, not a complete inventory of Nepal's monuments.
- The 3D models are procedural approximations built from a description of each site. They are not photogrammetry scans and do not claim survey accuracy.
- Quiz questions, clues and short descriptions were drafted with the help of an AI assistant from the collected facts. All Nepali text was produced by machine translation under a written style guide. Neither has been reviewed by a heritage expert or a native Nepali editor.
- Opening hours and fees come from public sources and can change. Walking times use a fixed detour factor and speed, and taxi fares are an estimate; there is no live traffic or fare data.
- Live hunt and game state is kept in server memory, so an unfinished game is lost on restart; a single server instance is assumed.
- The application was verified with automated tests and by the author's own use. No formal usability study has been carried out.
- The manual "I am here" stamp can be used by someone who is not on site; it is recorded as a manual stamp, and GPS check-in depends on device accuracy.

1.5 Significance

For students and the public, the project gives one place where heritage facts are shown with sources in a language of choice. Teachers get a bank of 1,160 explained questions and a live hunt a class can play on phones. Visitors get a schedule that fits a day. Smaller sites outside the Kathmandu Valley receive the same presentation as famous ones. For the author, it is a complete exercise in analysis, design, algorithms, real-time systems and 3D graphics, and the code and data are open for later extension.

1.6 Development Methodology

The project followed an iterative and incremental process: a small working core grew in cycles of planning, building, testing and review, each delivering a usable increment [4]. It suits this project because it is carried out by one student, the requirements became precise only after the dataset existed and could be played with, the riskiest parts (planner, real-time rooms, procedural 3D) could be tried early on small data, and each increment could be

tested on its own. Table 1.1 lists the increments; the tests of earlier areas were rerun before each new increment was accepted.

Table 1.1: Iterations of the development

Iteration	Increment delivered	Verified by
1	Schema, seed script, first 42 Kathmandu Valley sites, REST API, story map, accounts and administrator role	AT-02 to AT-04, AT-08, AT-09
2	Walk planner (orienteering with time windows), recommender, shareable walks	UT-01 to UT-05, AT-06
3	Adaptive quiz (Elo, SM-2), leaderboards, Find the Monument, passport and badges	UT-06 to UT-09, AT-10, AT-11
4	Live hunts over Socket.io; procedural 3D builder; Nepali interface; audio guide	hunt-sim script, visual review
5	Expansion to 116 sites, source and photo verification, e-mail verification, hardening	AT-01 to AT-18, dataset checks

1.7 Report Organization

Chapter 1 introduces the project, its objectives, scope and method. Chapter 2 presents the theory, reviews existing systems and related research, and states the research gap. Chapter 3 analyzes the system: requirements, feasibility, use cases, data and process models, and the dataset. Chapter 4 describes the design (architecture, database, algorithms, 3D pipeline). Chapter 5 describes the implementation and testing. Chapter 6 concludes and lists future work. References and appendices follow.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Theory

2.1.1 Digital Heritage and UNESCO Listing

Digital heritage is the use of computing to document, interpret and share cultural heritage, from databases and photographs to 3D reconstructions and serious games [5]. Sampada adopts the principle that play should rest on accurate content: the games sit on a sourced dataset, and every quiz answer comes with an explanation pointing back to a fact. The Kathmandu Valley property was inscribed in 1979 under criteria (iii), (iv) and (vi) [1]; the dataset flags 27 of its 116 entries as UNESCO sites, including Lumbini [2]. The "Sapta Sampada" badge is earned by stamping all seven valley zones.

2.1.2 Gamified Learning

Gamification is the use of game design elements in non-game contexts [6]. Empirical reviews report mostly positive effects that depend strongly on context and users [7], and a meta-analysis found small to moderate effects on cognitive, motivational and behavioral outcomes, with fiction and social interaction mattering for behavior [8]. Sampada uses points, levels, badges, leaderboards, the fiction of a pilgrim collecting stamps, and social play in live hunts. These choices follow the findings; they are not claimed to be proven for this application. A leaderboard alone is not enough, so explanations and site pages must carry real content.

2.1.3 Elo Rating and Adaptive Practice

The Elo system gives each player a rating and predicts the chance of winning against another as $1 / (1 + 10^{((R_B - R_A)/400)})$; after a game the rating moves by K times the difference between result and prediction [9]. Adaptive practice treats an answer as a match between learner and question, estimating both skill and difficulty on the fly [10]; the approach is simple, robust and effective in education, including geography facts that resemble heritage facts [11]. In Sampada the next question is the one whose predicted chance of a correct answer is closest to 70%, and K falls from 40 to 28 to 18 as a player answers more questions.

2.1.4 SM-2 Spaced Repetition

Memory fades unless material is reviewed at growing intervals. SM-2, the basis of SuperMemo, computes the next interval from an ease factor and the quality of the last recall [12]. Retrieval practice improves retention more than re-reading [13], and optimized schedules improve recall in large natural experiments [14]. Sampada uses the simpler SM-2 rule, which needs no training data, only for missed questions: a card starts at ease 2.5, returns after 1 day, then 6 days, then at intervals multiplied by the ease, which never falls below 1.3.

2.1.5 Orienteering Problem and Haversine Distance

In orienteering, a competitor visits control points within a time limit to collect the highest total score [15]. The problem is NP-hard, so large instances use heuristics [16]; with opening hours it becomes the time-window variant, and applied to sightseeing, the tourist trip design problem [17]. A heritage day plan is exactly this problem. Sampada uses greedy insertion by value per added minute followed by 2-opt, a local search that reverses route segments while they shorten the route [18]. Travel times come from the haversine formula, $d = 2R \cdot \operatorname{asin}(\sqrt{(\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cos\varphi_2 \sin^2(\Delta\lambda/2))})$ with R = 6,371 km, which is stable for short distances [19]; it also scores guesses and checks whether a device is within 60 meters of a site.

2.1.6 WebGL, Procedural Modeling and WebSockets

WebGL exposes the graphics processor to JavaScript without plugins [20]; three.js wraps it in a scene graph [21], and React Three Fiber expresses the scene as React components. Procedural modeling generates geometry from rules, first for cities [22] and then for buildings with shape grammars [23]. Newar pagodas suit it well: a repeated structure of roof tiers on a stepped plinth. Sampada describes each site with a small specification and one builder turns it into a scene. A WebSocket is a full-duplex channel opened by an HTTP handshake [24]; Sampada uses Socket.io on top of it for live hunts, rooms identified by a six-character code where the server owns the clock and the scores.

2.2 Review of Existing Systems

The comparison below is based on public pages and is limited to features relevant to this project. It states what each system does, not its quality.

- **Google Arts & Culture** offers high-resolution images, virtual tours and curated stories from partner institutions [25]. Small Nepali shrines appear thinly represented, and the author found no day planner or live local hunt.
- **UNESCO World Heritage Centre** is the authoritative source for listed properties [1], but it is not interactive and gives no visiting or planning help.
- **Department of Archaeology and tourism resources** hold the legal record of monuments [26] as lists and books, not as a learning or planning tool.
- **Wikipedia, Wikidata and Commons** are a large free multilingual source [27] on which Sampada itself depends, but they have no common structure for Nepali sites and uneven quality.
- **Sketchfab** hosts precise scans, including post-earthquake Kathmandu [28], but for few monuments, each on a separate page, and some are sold.
- **Location-based games such as Pokémon GO** bring people to real places, but their points of interest carry little verified heritage content.

Table 2.1: Comparison of Sampada with existing systems

System	Nepal-wide small sites	Sourced facts	Day planning	3D model	Games	Nepali
Google Arts & Culture	Limited	Curated	No	Some	Generic	Limited
UNESCO WHC site	Listed only	Official	No	No	No	No
Dept. of Archaeology	Lists	Official status	No	No	No	Mixed
Wikipedia and Commons	Uneven	Varies	No	No	No	Some
Sketchfab	Few	Notes	No	Scanned	No	No
Pokémon GO style	No	No	No	No	Yes	No
Sampada	116 sites, 7 provinces	568 source links	Yes, with hours	Procedural, all 116	Quiz, guess, hunts	Machine-written, unreviewed

Sampada does not replace these systems and is weaker in several respects: it can never be more authoritative than its sources (Commons, Wikipedia, Wikidata, OpenStreetMap), its 3D models cannot match a scan, its dataset is small, its Nepali text is unreviewed, and it has not been tested at large scale. What it offers instead is a combination none of them provides: a structured dataset of small and large sites across all provinces, an opening-

hours-aware day planner, an adaptive quiz, a distance-scored guessing game, GPS-checked passport stamps, account-free live rooms and a uniform 3D model of every site, in one free bilingual interface.

2.3 Review of Related Research

Hamari et al. found gamification effects mostly positive but context dependent, with a risk of novelty effects [7], which supports tying games to real content. Klinkenberg et al. built a large adaptive arithmetic system on Elo without a calibration phase [10], and Pelánek recommended Elo for its simplicity [11]. For tour planning, surveys conclude that heuristics are the practical choice for interactive planners [16], [17]; Sampada's planner is a compact member of this family, tested against exhaustive search on small instances (UT-05) but with no optimality guarantee on large ones. Procedural modeling systems were built for general urban models, not Nepali styles [22], [23], while 3D documentation of Nepali monuments relies on scanning [28]; a parameterized builder is a lower-cost route that needs no survey of each site. Wikidata and OpenStreetMap supply structured and geographic data [27], [29] but are only as accurate as their last editor; Section 3.6 describes the cross-checks and discrepancies found.

2.4 Research Gap

1. Scanned 3D models exist for only a few monuments; there is no low-cost uniform 3D view of many sites across the country.
2. Heritage platforms for Nepal are not combined with planning that understands opening hours, walking and taxi legs and interests.
3. Adaptive quiz methods well studied in mathematics and geography have not, to the author's knowledge, been applied to Nepali heritage content.
4. GPS-verified visits tied to verified content and a passport are not widely available, and bilingual, openly licensed content covering all seven provinces is missing.

Sampada addresses these gaps at the scale of a student project. It claims no new algorithms; its contribution is the integration of established techniques over one carefully collected dataset, with an honest account of where the result is approximate.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Requirement Identification

The stakeholders are the guest visitor, the registered explorer (who keeps a rating, review cards, stamps, badges and saved walks), students and teachers using the quiz and hunts, the administrator who maintains the dataset, and heritage reviewers who are not yet involved (Section 1.4.2). Requirements were collected from the systems of Section 2.2, from user stories per stakeholder, and refined over the iterations of Table 1.1.

3.1.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement	Actor
FR-01	Browse and search sites; filter by province, type, religion, city and UNESCO status	All
FR-02	Site page with story, legend, facts, festivals, hours, fees, 2015 earthquake note, gallery and sources	All
FR-03	Interactive 3D model of each site with day and dusk lighting	All
FR-04	Nepal map with province filters and clustered pins	All
FR-05	Audio guide; English and Nepali interface and content	All
FR-06	Recommend sites with stated reasons	All
FR-07	Register, verify e-mail by code, sign in, reset password	Visitor
FR-08	Plan a heritage walk from start point, time window and interests; save and share by link	All; saving needs sign-in
FR-09	Adaptive quiz choosing questions by rating, with explanations; spaced-repetition queue of missed questions	All; queue for Explorer
FR-10	Find the Monument: guess a location from a photo and clue with hints and server-side scoring	All
FR-11	Host and join live hunts by six-character code without an account, with live leaderboard and reconnection	All
FR-12	Stamp a site by GPS (within 60 m) or manually; award badges; favorites, ratings and leaderboards	Explorer
FR-13	Send verification and reset e-mail through SMTP, or save a preview when SMTP is not configured	System
FR-14	Administrator statistics; create, edit and delete sites, questions and clues; upload photographs	Administrator
FR-15	Deactivate users, change roles, delete reviews	Administrator

3.1.2 Non-functional Requirements

Table 3.2: Non-functional requirements

ID	Quality	Requirement
NFR-01	Security	bcrypt password hashes; signed JWT valid 7 days; admin role check on every admin route; e-mail codes expire in 10 minutes, stored as HMAC hashes, lock after 5 wrong tries
NFR-02	Security	Sign-in paused for five minutes after ten wrong passwords; image uploads only, by file signature, up to 6 MB; request bodies up to 200 KB
NFR-03	Fairness	Answers never sent to the client before answering; option order shuffled per run; time measured by the server
NFR-04	Performance	Catalog served from memory, refreshed every 60 seconds or on admin edit; day plan within a few seconds
NFR-05	Usability	Responsive layout; light, dark and high-contrast themes; keyboard focus visible; English and Nepali
NFR-06	Reliability	Live players keep their place for a grace period after a dropped connection; unused rooms are cleaned up
NFR-07	Maintainability	Algorithms are pure modules separate from routes; 12 unit and 18 API tests; data in JSON files seeded by one script
NFR-08	Legal and cost	Photographs only under Creative Commons or public-domain terms with credit shown; free components and tiles, no paid API keys

3.2 Feasibility Study

Technical. Every component is open source and runs on an ordinary laptop: Node.js, Express 4, Socket.io 4, MySQL 8.4, React 18, Vite 5, three.js with React Three Fiber, and MapLibre GL. The risky parts were prototyped in early iterations (Table 1.1), and a portable MySQL copy with a setup script removes installation difficulty. **Operational.** The system needs no training; guests can start a quiz at once and a live hunt needs only a code and a name. Operation needs one person to keep the server running and review content.

Economic. Table 3.3 gives rounded estimates by the author, excluding the author's own time. Running cost is dominated by hosting, and the system can also run free on a college computer; the benefit is non-monetary.

Table 3.3: Estimated cost of development and a first year of small deployment

Item	Estimated cost (NPR)
Software, libraries, map tiles, photographs, textures, fonts, Gmail SMTP	0 (free and open source)
Internet during about five months of development	7,500
Small virtual server for one year (1 vCPU, 1 to 2 GB RAM)	About 8,000

Item	Estimated cost (NPR)
Domain name for one year (optional)	About 1,800
Printing and binding of the report	About 2,500
Total	About 19,800

Schedule. Work was planned as the five iterations of Table 1.1, with data collection parallel to the early increments and report writing at the end (Figure 3.1). Because every iteration produced a working increment, a delay in one area, such as the 3D models, did not block the others. The project is feasible within the eighth semester.

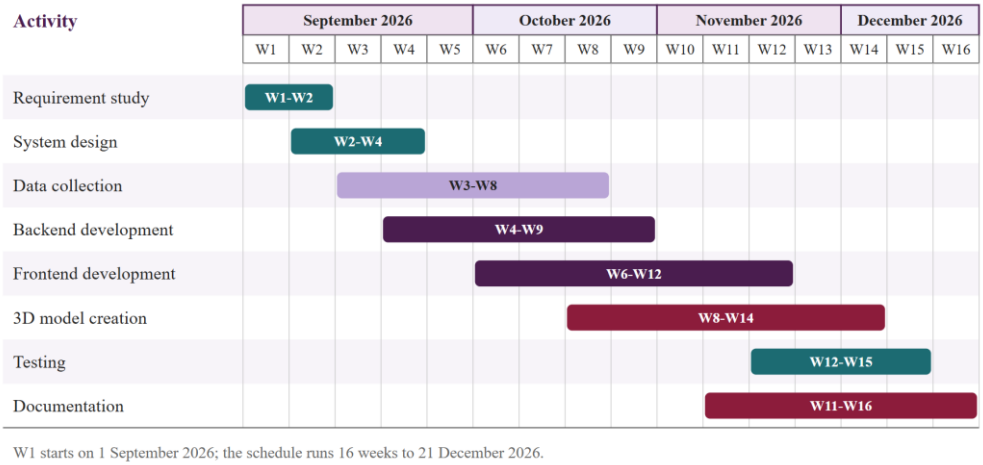


Figure 3.1: Project schedule (Gantt chart)

3.3 Use Case Model

Figure 3.2 shows the actors and use cases. A guest can explore and play; an explorer is a registered guest who also keeps progress, stamps and saved walks; the administrator maintains content and users; the mail server is a secondary actor that receives verification and reset messages.

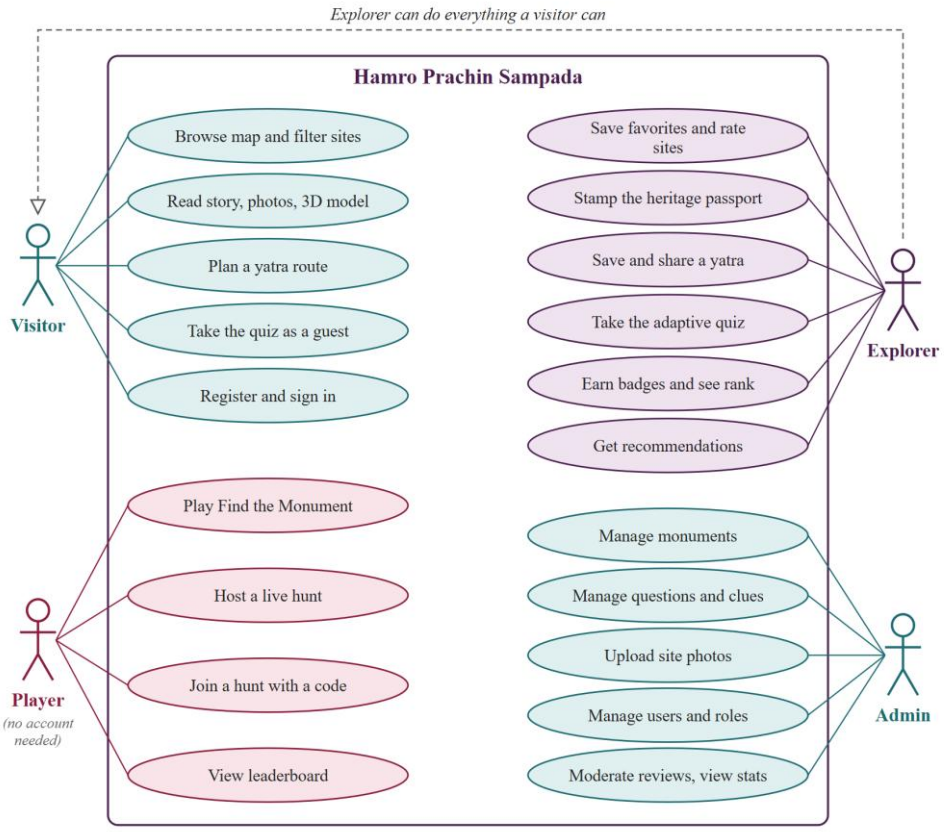


Figure 3.2: Use case diagram of Hamro Prachin Sampada

Two of the central use cases are described in Tables 3.4 and 3.5.

Table 3.4: Use case description: Plan a heritage walk

Item	Description
Actor	Guest or explorer
Main flow	1. Choose a start point and set start and end time, interests and optional must-see sites. 2. System computes site values, builds travel legs (walk up to 2.5 km, otherwise taxi) and runs greedy insertion and 2-opt. 3. System shows ordered stops with arrival and leave times, waiting time and fare estimate.
Alternative flow	An end time not after the start is rejected. A closed or too distant site is listed as skipped with a reason.
Postcondition	A signed-in explorer may save the walk and share it by a ten-character link

Table 3.5: Use case description: Answer adaptive quiz questions

Item	Description
Actor	Guest or explorer

Item	Description
Main flow	1. System picks the unseen question whose predicted success is closest to 70%, preferring questions due for review. 2. Player answers within 25 seconds; options are shuffled. 3. System checks the answer on the server, updates player and question ratings by Elo, and returns the answer with an explanation. 4. A wrong answer creates or resets a review card (SM-2).
Alternative flow	An answer after the limit plus a 3 second grace counts as a time-out; a question already answered in the run is refused.
Postcondition	Rating, attempt and review card are saved for an explorer; a guest keeps the rating in the browser

3.4 Data Modeling

The database has 14 tables (Figure 3.3, Table 3.6). The `monuments` table is the center: questions, clues, visits, favorites and ratings all refer to it.



PK primary key, FK foreign key, UK unique key. 1 and N mark the one and many sides of each relationship. email_codes is tied to users by email address, and hunts to game_scores by hunt_code, without foreign keys.

Figure 3.3: Entity relationship diagram

Table 3.6: Entities and their main attributes

Entity	Key attributes and relationships
users, email_codes	email (unique), password_hash, role, interests, Elo rating, is_active, email_verified; one-time code hashes with purpose, attempts, expiry
monuments	slug (unique), English and Nepali names, province, lat, lng, type, story, legend, facts, hours, fees, photo and credit, sources, 3D model (JSON columns for lists)
questions, clues	monument_id (FK); question, four options, answer_index, difficulty, Elo rating, explanation, Nepali counterparts; clue text
quiz_attempts, review_cards	user and question (FK); correctness, time, rating before and after; SM-2 ease, interval, due date
visits, favorites, ratings, badges	user and monument (composite keys); stamp method (gps, manual, game); stars 1 to 5; badge key

Entity	Key attributes and relationships
tours, game_scores, hunts	saved walk with plan JSON and share code; solo and hunt scores (user nullable, SET NULL); finished hunt results

Lists read together with a site (facts, festivals, gallery, model, sources) are stored as MySQL JSON columns instead of many small tables. A zone such as a Durbar Square is a monument whose slug other monuments name in `zone_slug`. Rows belonging to a user are deleted with the user (`ON DELETE CASCADE`), but a game score keeps the score and drops the link, so leaderboards do not change when an account is removed.

3.5 Process Modeling

At context level (DFD level 0), Sampada is a single process. The guest or explorer sends searches, answers, guesses, locations and walk requests and receives pages, questions, scores, plans and stamps; the administrator sends content and user changes and receives statistics; the system sends e-mail to the mail server and reads map tiles and photographs from external services. Figure 3.4 splits it into catalog and discovery, walk planning, adaptive quiz, games and live hunts, passport and badges, accounts and e-mail, and administration, over four data stores: catalog, users and codes, progress, and saved tours. The live hunt keeps room state in memory and writes only the final result.

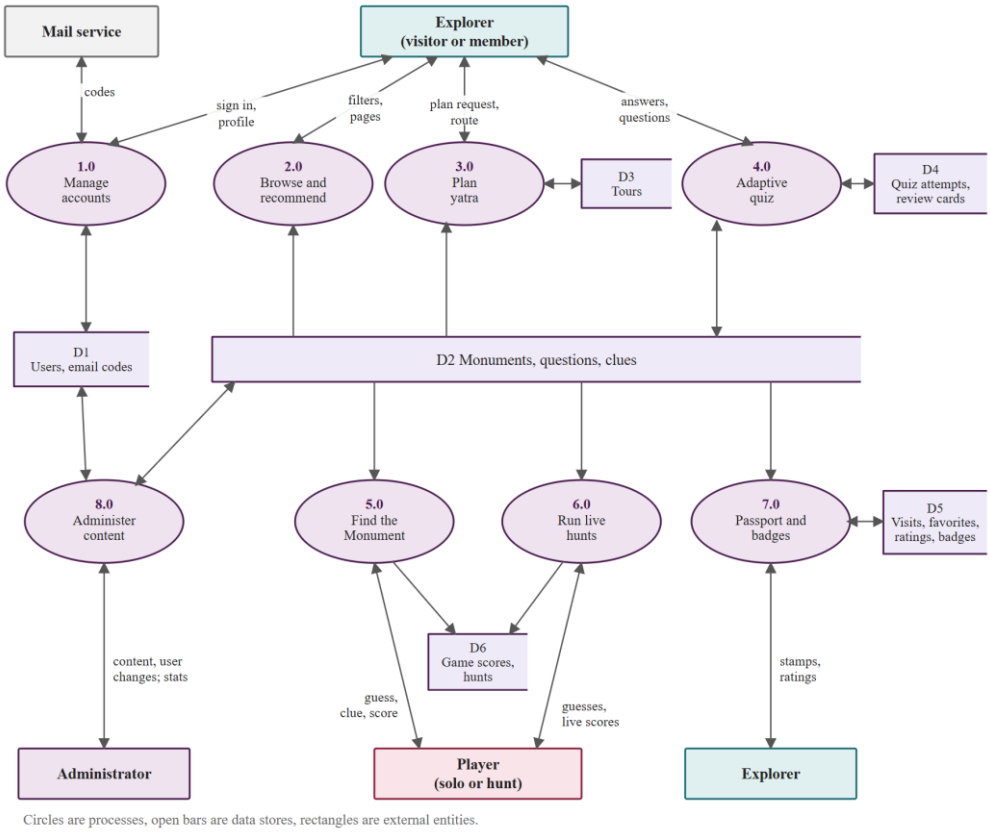


Figure 3.4: DFD level 1

3.6 Data Collection and Verification

The quality of Sampada depends on its dataset, so it was collected in stages, each with a checkable source. This section states what has and has not been verified.

3.6.1 Selection of Sites

The first set was the seven UNESCO zones of the Kathmandu Valley with their principal monuments; it was then extended to the other six provinces by choosing recognized pilgrimage places, forts, archaeological sites and protected monuments that have a free licensed photograph and enough published information (Table 3.7). The imbalance toward Bagmati is deliberate for this version and is a stated limitation. By type there are 46 pagoda temples, 9 shikhara temples, 7 palaces, 7 courtyards, 6 stupas, 5 forts and others.

Table 3.7: Sites in the dataset by province

Province	Sites	Province	Sites
Bagmati	62	Lumbini	11
Gandaki	15	Koshi	7
Sudurpashchim	10	Madhesh	6
Karnali	5	Total	116

3.6.2 Coordinates, Facts and Photographs

The preferred coordinate source is the OpenStreetMap centroid [29], the second the Wikidata property P625 [27]; where both exist they were compared and the source is noted in coord_source. They disagree often enough to matter: one Wikidata coordinate for a valley temple pointed more than 100 km away, several had only two or three decimals (about a kilometer), and one item carried the coordinates of a town in India. In such cases the OpenStreetMap point or a geotagged photograph was used and the reason noted. Two entries are marked “approximate”, and the 13 Patan entries list OpenStreetMap or Wikidata links among their sources but carry no separate coord_source note. Coordinates have been compared by source only, not surveyed in the field.

Each fact sheet (description, story, legend, built year, dynasty, architecture, deity, festivals, 2015 earthquake effect) was written from published sources listed on the site page: 568 links in all, 3 to 8 per site, mostly Wikipedia (143), OpenStreetMap (81), Wikidata (76), Wikimedia Commons (69) and UNESCO (15). Opening hours are recorded for 10 sites and fees for 43; for other sites the planner assumes the site is always open, and the site page says that hours and fees change. There are 708 photographs, all from Wikimedia Commons with author and license stored and shown; the main photographs are mostly CC BY-SA 4.0 (94 of 116). Textures are CC0 from Poly Haven. Each site has a 3D model specification written from photographs and descriptions, in 15 model kinds, the largest being pagoda (49 sites).

3.6.3 Quiz Questions, Clues and Nepali Text

Each site has exactly 10 multiple-choice questions (1,160 in all, difficulty 1 to 5, each with four options and a one-sentence explanation) and 6 clues (696 in all). The questions and clues were drafted with the help of an AI assistant from the fact sheet of the site, with the instruction that each answer must be supported by that sheet. Stored answer positions are not balanced, so the server shuffles options per run before comparing the choice. The Nepali names, descriptions, questions, clues and interface strings were produced by machine translation under a style guide asking for everyday polite Nepali; all 116 sites and 1,160 questions have a Nepali version, which has not been reviewed by a native editor and may contain unnatural wording or spelling mistakes.

3.6.4 Checks Performed and Their Limits

Every site has coordinates, a main photograph with a stored license, a model specification, a Nepali version, at least 3 source links, and exactly 10 questions of four options with the answer index in range. MySQL enforces column types at seeding, the administrator API validates edits, and test AT-02 confirms that the catalog has more than 100 sites in all seven provinces.

The following have not been done. The facts have not been reviewed by a historian or the Department of Archaeology and inherit the errors of their sources. The AI-drafted questions and clues may contain mistakes that the checks cannot detect, since a check can confirm that an answer exists but not that it is true. The Nepali text has not been proofread by a native speaker. The coordinates have not been verified on the ground. Until these reviews are done, the dataset should be described as researched and cross-checked, not officially verified. The administrator can correct entries at any time (FR-14).

CHAPTER 4: SYSTEM DESIGN

This chapter turns the analysis of Chapter 3 (System Analysis) into a design: architecture, database, algorithms, the 3D model pipeline, the interface, the API and security. Every parameter quoted is taken from the source code.

4.1 Architecture Design

Hamro Prachin Sampada is a three-tier web application with a live-play channel. The presentation tier is a React 18 single page application built with Vite. The application tier is one Node.js process running an Express 4 REST API and a Socket.io 4 server on the same port (5050). The data tier is MySQL 8.4, used through a `mysql2` pool of at most ten connections. The browser uses JSON over HTTP, and a WebSocket (long polling as fallback) for hunts.

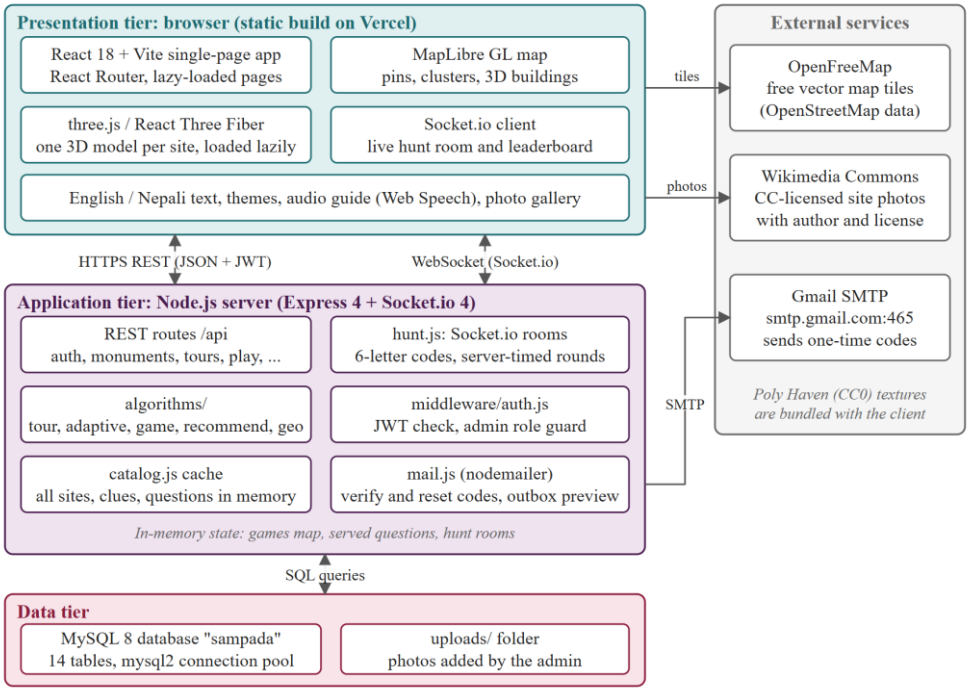


Figure 4.1: System architecture of Hamro Prachin Sampada

Three decisions shape the system. First, the catalog (116 sites, 1,160 questions, 696 clues) is cached in memory by `catalog.js` and refreshed after 60 seconds or an administrator edit, so a catalog read or quiz question needs no query. Second, everything secret during play, such as the true location of a round, stays on the server. Third, the algorithms are pure functions in `server/src/algorithms` (`geo`, `tour`, `adaptive`, `game`, `recommend`) that touch neither database nor network, so the twelve unit tests run in about 200 ms.

The system is demonstrated on one Windows computer: MySQL (portable copy, port 3307), the API (5050) and the Vite server (5174), which proxies `/api`, `/uploads` and `/socket.io` and is reachable from phones on the same Wi-Fi so a hunt can be played in a classroom. A public release needs static client files and one Node.js process with one MySQL database (Section 5.6 explains why one process).

4.2 Database Design

The schema has 14 tables (MySQL 8, InnoDB, utf8mb4 for Nepali text). Descriptive fields always read with their site are JSON columns (facts, festivals, gallery, sources, 3D `model`), avoiding many join tables; data queried by user has its own table (Figure 4.2, Table 4.1).



PK primary key, FK foreign key, UK unique key. 1 and N mark the one and many sides of each relationship. email_codes is tied to users by email address, and hunts to game_scores by hunt_code, without foreign keys.

Figure 4.2: Entity relationship diagram

Table 4.1: Tables of the database

Table	Purpose and main columns
users	Accounts: email (unique), password_hash, role, interests, Elo rating (default 1000), is_active, email_verified
email_codes	Verify and reset codes: code_hash (HMAC), attempts, expires_at
monuments	One row per site: slug, English and Nepali text, lat, lng, type, district, UNESCO flag, hours, fees, facts, gallery, model (JSON)
questions	10 per site: four options (JSON), answer_index, difficulty 1–5, Elo rating, explanation, Nepali text
clues	Riddles for map games: clue, clue_np
quiz_attempts	Each signed-in answer: correct, time_ms, rating before and after
review_cards	SM-2 schedule: ease, interval_days, reps, due_at
visits	Passport stamps: method (gps, manual, game)
favorites	Saved sites per user
ratings	One star rating (1–5) and comment per user and site
tours	Saved walks: input_json, plan_json, unique share_code
game_scores	Finished solo and hunt games: points, rounds (user null for guests)
hunts	Archive of finished hunts: code, players, winner, results_json
badges	Badges earned: user_id and badge_key

Child tables reference their parent with `ON DELETE CASCADE`. Unique keys prevent duplicates (email, slug, one rating and one stamp per user and site, one hunt code), and rating and stamp writes use `INSERT ... ON DUPLICATE KEY UPDATE`, so a repeated request updates instead of failing. Passwords are stored only as bcrypt hashes and email codes only as HMAC digests.

4.3 Algorithm Design

The algorithms are written without libraries in `server/src/algorithms` and `server/src/hunt.js`.

4.3.1 Distance and Travel Model

Distances use the haversine formula [19] with R = 6,371,000 m:

```
a = sin²(Δφ/2) + cos φ1 · cos φ2 · sin²(Δλ/2)
d = 2 · R · asin(√a)
```

Walking distance is the straight distance times 1.35. A leg is walked if it is at most 2,500 m, at 4.2 km/h. A longer leg is a taxi: distance times 1.45, 16 km/h, 8 minutes of waiting, and a fare of Rs 150 plus Rs 60 per km rounded to Rs 10. These are named constants in `geo.js`, and `legMatrix` computes every leg once per request.

4.3.2 Yatra Planner

Choosing which sites to visit, and in what order, so that total interest is highest and the day fits the opening hours, is the Orienteering Problem with Time Windows [16]. It is NP-hard, so the planner uses a heuristic that always returns a feasible plan (Figure 4.3).

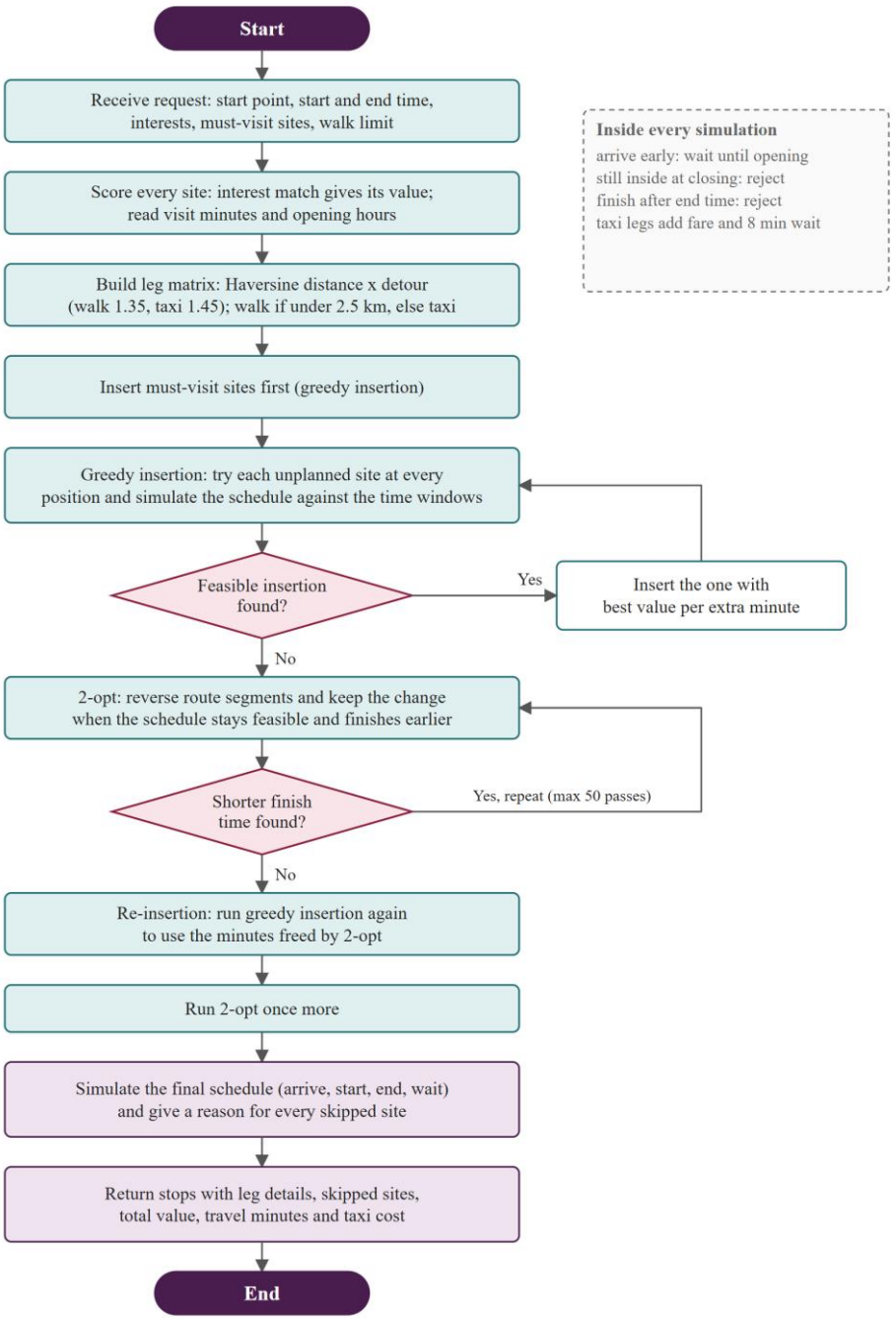


Figure 4.3: Flow of the yatra planner

Value and feasibility. Inputs are a start point, start and end time (default 09:00–17:00), and optional chosen sites and interests. Each site has a visit time (`visit_minutes`, 30 if missing), hours, and a value of 1 + 3 × cosine(interests, tags) + 0.5 for UNESCO + 4 if

chosen; chosen sites are must-see, and with none chosen the candidates are the sites within 25 km. `simulate(order)` adds each leg time, waits if a site is not yet open, and rejects a route if a visit would run past closing or the finish passes the end time. It costs O(n) and every step calls it, so no step can break a window.

Search. `greedyInsert` tries every unvisited site at every position, keeps the feasible insertions and takes the highest value per extra minute (the increase of the finish time, at least one). The route is then improved by 2-opt [18]: a segment is reversed if the route stays feasible and finishes earlier, for up to 50 passes. The freed minutes are spent by a second insertion and a second 2-opt pass. Sites left out are reported with a reason (closed in the window, too far, or did not fit with higher-value stops).

Complexity. The leg matrix is $O(n^2)$, one insertion round $O(n^3)$ and at most n rounds are needed, so the worst case is $O(n^4)$; one 2-opt pass is $O(n^3)$. Real routes have 5 to 20 stops (measured in Section 5.5). `bruteForce` enumerates all subsets and orders for up to eight sites and is the exact baseline in tests.

4.3.3 Adaptive Quiz

Each player has a skill rating and each question a difficulty rating, following the Elo model [9]. The chance that a player of rating Rp answers a question of rating Rq correctly is:

```
E = 1 / (1 + 10 ^ ((Rq - Rp) / 400))
```

A player starts at 1,000 and a question at 800 + (d − 1) × 150 for difficulty d = 1 to 5. After an answer the player rating changes by K × speed × (S − E), with S = 1 for correct and 0 for wrong. K is 40 for the first ten answers, 28 up to thirty and 18 after, and speed = 1 + 0.15 × max(0, 1 − t/25 s) rewards fast answers. The question rating moves by half the player change in the opposite direction; both stay between 600 and 1,800 (Figure 4.4).

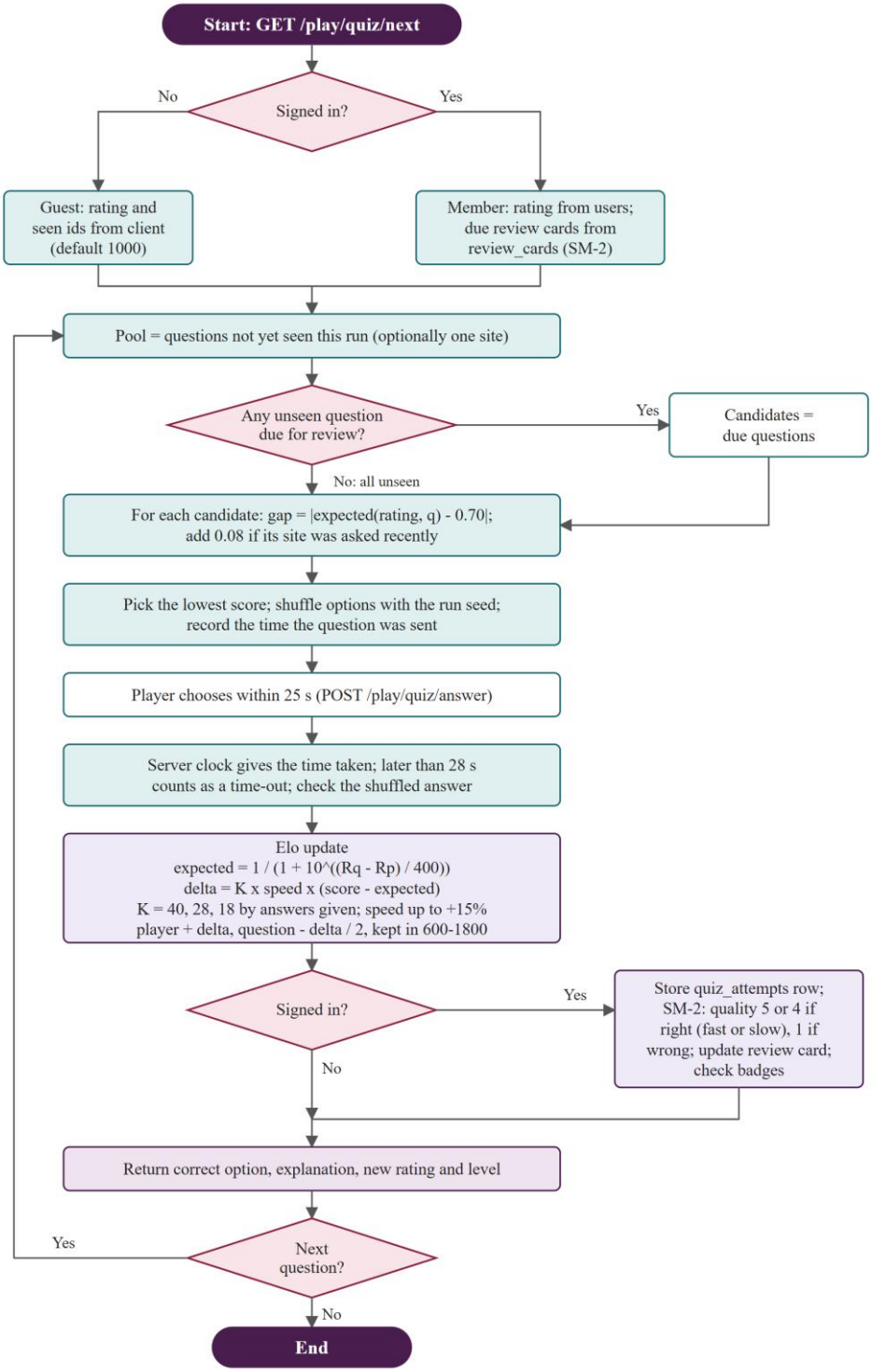


Figure 4.4: Flow of the adaptive quiz and spaced repetition

Choosing and reviewing. `pickNext` selects the unseen question whose predicted success is closest to 0.7, which keeps the quiz challenging but not discouraging; a question from the same site as a recent one is penalized by 0.08, and if any review is due only those are considered. A question a signed-in player misses enters an SM-2 schedule [30]: quality is 5 for a correct answer faster than a third of the time limit, 4 for a slower correct one and 1

for a wrong one. Quality below 3 resets repetitions and the interval to one day; otherwise the interval is 1 day, 6 days, then the previous interval times the ease factor. Ease starts at 2.5, changes by $0.1 - (5 - q) \times (0.08 + (5 - q) \times 0.02)$ and never goes below 1.3. The rating maps to levels from New Explorer (below 960) to Sampada Guru (from 1,350). Guests keep their rating in the browser; only signed-in players have a stored rating and review queue.

4.3.4 Find the Monument and Live Hunts

The player sees a photo and a riddle, drops a pin on a map, and scores for closeness and speed:

```
distance = 800                              if d ≤ perfect
         = round(800 · exp(-(d - perfect)/scale))   otherwise
speed    = round(200 · (1 - t/limit) · distance/800)
points   = max(0, min(1000, distance + speed - 100 · hints))
```

The curve falls smoothly, so a near miss still scores, and speed is scaled by the distance part, so a fast wrong pin earns nothing extra. A hint costs 100 points and reveals a second clue, the city and the type. In the Valley scope a pin within 40 m is perfect and the scale is 1,200 m (a 500 m miss scores about 545 distance points, a 5 km miss about 13); in the Nepal scope the radius is 1,000 m and the scale 40,000 m (a 5 km miss scores about 724). A round lasts 60 seconds; no pin scores zero.

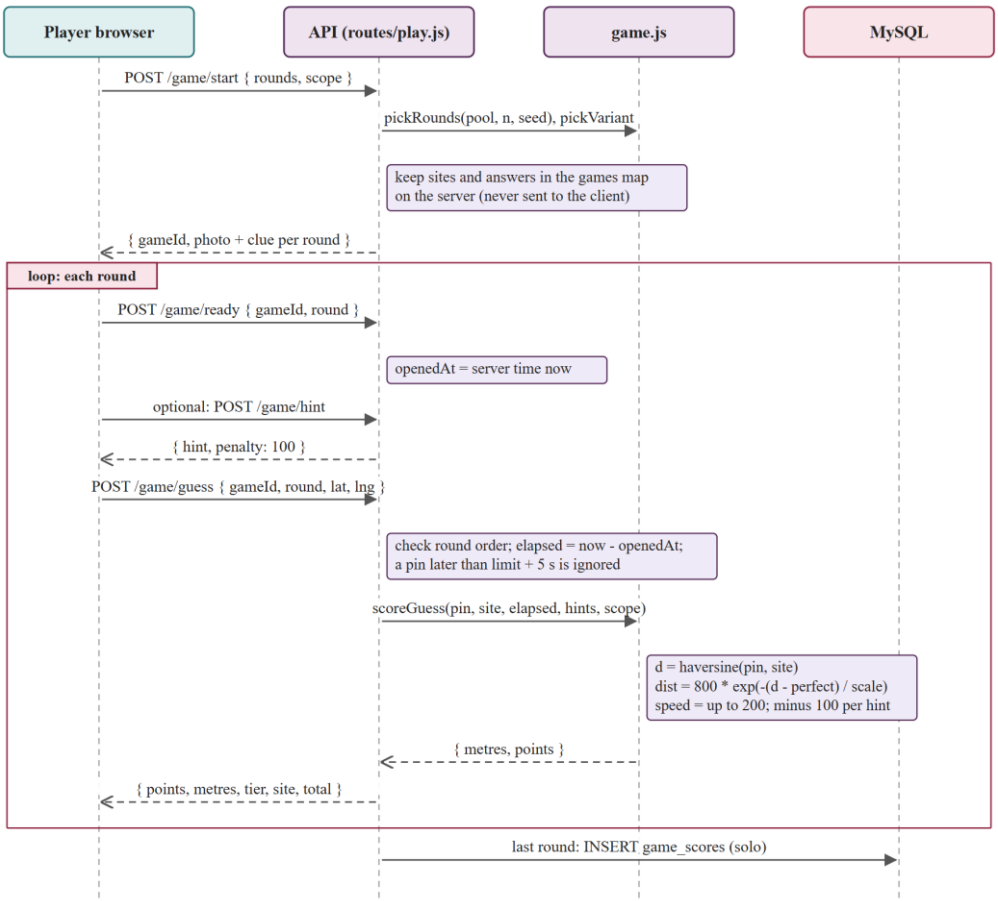


Figure 4.5: Sequence of one Find the Monument game

The answer stays on the server until a guess is scored (Figure 4.5). `POST /api/play/game/start` picks rounds with a seeded shuffle, prefers clues and photos the player has not seen, and returns only photo, clue and credit. The client calls `/game/ready` when a round appears, which starts the server clock, and a pin is accepted up to five seconds after the limit to allow for network delay.

Live hunts. A hunt lets a group play the same rounds on their phones. A room has a six-character code from 32 symbols without 0, O, 1 and I, 3 to 10 rounds (default 5) of 20 to 90 seconds (default 45), up to 30 players, and photos from the Kathmandu Valley. The host creates a room, players join by code, and the host starts and advances rounds; a guess is one pin per round, with a reply of points but never the answer, and the answer arrives only in the reveal with every guess. The server ends a round with a timer at the limit plus 300 ms, or earlier when everyone has answered, refuses guesses later than the limit plus 400 ms, and sends its clock (`serverNow`) so a countdown is right even if a phone clock is wrong.

Robustness. The answered flag is set before any asynchronous work, so a double tap cannot score twice. Players are identified by a stable id, not the socket, so a refresh rejoins the same seat; a dropped player keeps the seat for 8 seconds, and if the host leaves the oldest connected player becomes host. Players are ranked by points, then total time (less is better), then join time; equal points and time share a rank. At the end one row goes to `hunts` and one per player to `game_scores`, and badges are checked.

4.3.5 Recommender

The recommender scores every site and shows the top six with reasons. The score is a weighted sum of interest (cosine similarity of binary tag vectors, $|A \cap B| / \sqrt{(|A| \cdot |B|)}$), rating (Bayesian average $(n \cdot avg + 5 \cdot mean) / (n + 5)$), novelty (1 if not yet stamped) and nearness ($1 / (1 + km / 2)$ when a position is known). With interests the weights are 0.45, 0.25, 0.15 and 0.15; without interests 0, 0.55, 0.20 and 0.25; without a position the nearness weight moves to rating. The Bayesian average stops one five-star review from outranking a hundred reviews of 4.8. The list is re-ranked for diversity in the spirit of maximal marginal relevance [31]: an item from a city already in the list loses 12 percent of its score for each earlier pick from that city. Each result carries reasons such as “matches your interests” or “450 m away”.

4.4 3D Model Design

Each of the 116 sites has an interactive 3D model with day and dusk-aarti lighting, drawn in code from a short specification and not loaded from a modeling program (Figure 4.6).

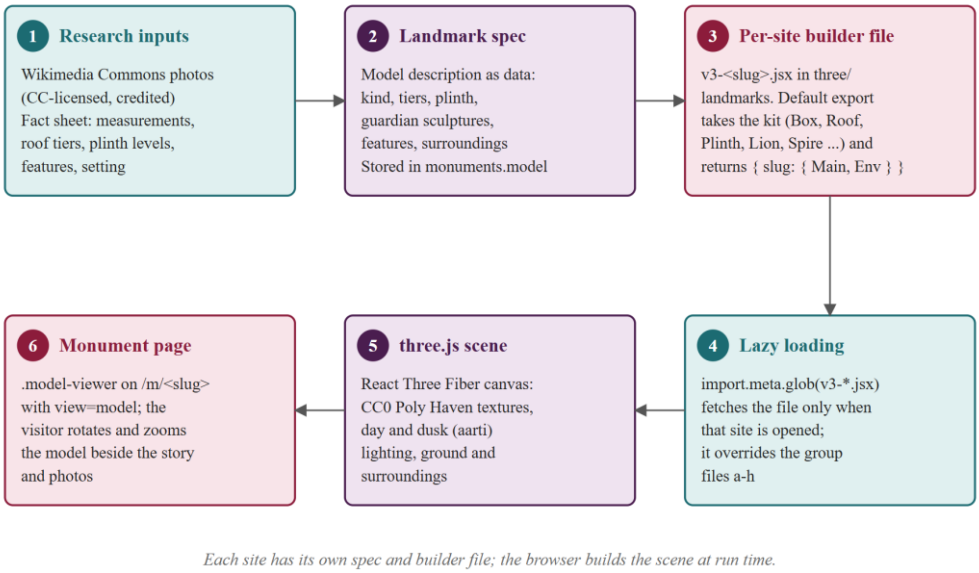


Figure 4.6: Pipeline from research notes to a 3D scene

Specification and toolkit. The `model` column holds a JSON specification: the kind (pagoda, stupa, shikhara, palace, square, pond, gate, fort, cave and others), roof tiers, plinth steps, footprint in meters, materials, features (guardian pairs, Garuda pillar, prayer wheels) and surroundings (plaza, Newar houses, river). A generic composer, `Monument`, builds any site from it. For faithful models, one file per site, `v3-<slug>.jsx`, receives a toolkit of about 70 shared parts (materials, roofs, sculptures, houses, water, trees) and returns the building and its surroundings, so all sites share one visual language and a fix in a shared part improves every site.

Lazy loading and lighting. The 116 site files hold about 68,600 lines and 2.3 MB of JavaScript (808 KB gzip). Vite splits each file into its own chunk and the viewer fetches only the opened site (median 19.6 KB, largest 35.5 KB for Pashupatinath); an error boundary falls back to the generic model if a file throws. By day the sun is warm white at intensity 3; at dusk the key light is orange at 1.8 with a violet fill, oil lamps (diyo) are lit and sparks drift.

Why procedural low-poly. Scanned models of 116 sites are not freely available, and scans of tens of megabytes suit neither mobile data nor a solo project. A specification lets a curator correct a roof count or a wall color without a 3D artist,. Surfaces use ten CC0 textures (7 MB).

4.5 User Interface Design

Palette and themes. The design uses four color pairs: Plum (#381932) text and deep surface, Milk (#FFF3E6) page, Wine Red (#81021F) main action, Persian (#27187E) links, and Lilac on dark surfaces. Colors are CSS custom properties in `tokens.css` with semantic names (`--bg`, `--text`, `--primary`). Recorded contrast ratios include plum on milk 15.6:1 and wine on milk 10.6:1; Lilac on milk is only 2.6:1 and is never body text. There are three themes applied through `data-theme`: Light, Ratri (dark, chosen first if the system is dark) and Dhoop (white background, black text, stronger borders and larger text for strong sunlight at a site).

Bilingual design. Every public page works in English and Nepali. The data has Nepali fields for names, stories, facts, questions, options and clues, and `i18n.js` translates the interface live from a dictionary: with the Nepali toggle on it swaps every known text node, placeholder, title, aria-label and image alt (including text React draws later) and restores English exactly.

Navigation and accessibility. The client has 16 pages: Home, Explore, site page (`/m/:slug`), Walk (and shared walk), Quiz, Find, Hunt, Leaderboard, About, Credits, account pages, Passport and Admin. Public pages need no login, because a visitor at a temple should not need an account to read about it; stamps, saved walks, ratings and favorites need an account, and `/admin` the administrator role. Accessibility measures include visible focus rings, a photo viewer that traps focus and closes with Escape, a `prefers-reduced-motion` check, translated alt texts, an audio guide through browser speech synthesis, and the Dhoop theme. They have not been audited with screen reader users (Section 5.6).

4.6 API Design

The REST API has 53 endpoints in six routers under `/api`, plus a health check and a LAN-address helper for the hunt QR code (Table 4.2). Bodies are JSON; a failure returns `{ "error": message }` with 400, 401, 403, 404, 409 (conflict), 415 (bad upload type), 422 (GPS stamp too far), 429 (too many attempts) or 503; a server error returns only "Something went wrong" and the details go to the log. `optionalAuth` attaches the user when a valid token is present without blocking, `requireAuth` guards personal data, and the whole admin router sits behind `requireAdmin`. Live hunts use Socket.io events (`host`,

`join`, `start`, `guess`, `reaction`, `next`, `again` from the client; `room`, `round`, `countdown`, `reveal`, `finished` from the server) instead of REST.

Table 4.2: REST endpoints by router

Router	Count	Main endpoints
/api/auth	10	register, verify-email, resend-code, forgot, reset, login; GET and PUT /me; interests
/api/monuments	6	search and filters; /meta, /recommend, /:slug; favorite, rate
/api/tours	5	POST /plan (no login); save, list, shared/:code, delete
/api/play	7	quiz/next, quiz/answer; game/start, ready, hint, guess; leaderboard
/api/passport	4	summary; add and remove stamp; GPS check
/api/admin	21	stats; monuments, questions, clues (CRUD); users, reviews, stamps, games, hunts; uploads

4.7 Security Design

Table 4.3: Threats and measures

Threat	Measure in the system
Stolen database	bcrypt hashes with cost 10 [33]; email codes stored only as HMAC-SHA-256 digests keyed with the server secret.
Forged or stale session	JSON Web Token [32] signed with `JWT_SECRET`, valid 7 days; an `is_active` flag cached for 30 s means a deactivated account loses access within 30 seconds.
Guessing	Ten wrong passwords for one email from one address pause login for five minutes (429); a six-digit code expires in 10 minutes, locks after 5 tries, is compared in constant time, and “forgot password” answers the same whether or not the account exists.
Cheating in games	The server holds true locations, enforces round order, one answer per round and the clock. Quiz options are shuffled per run by a seed, so the stored order is never sent.
Injection and upload	Bound SQL parameters; whitelisted admin columns; JSON bodies limited to 200 KB. Photo upload (6 MB) detects JPEG, PNG or WebP from the first bytes, names the file by SHA-256 and serves it with `nosniff`.
Cross-origin and privilege misuse	CORS admits only the configured client origin and private LAN addresses. Admin routes need the admin role re-read from the database; ratings, favorites and stamps always use the id from the token.
Secret leakage	Secrets are read from `server/.env`, which is not committed; only `.env.example` is.

Known weaknesses are in Section 5.6: no general per-address rate limit, a development fallback secret if `JWT_SECRET` is unset, and a token in local storage.

CHAPTER 5: IMPLEMENTATION AND TESTING

This chapter describes the tools, the main modules, the screens and the testing. Measurements were taken on the development computer (Windows 11, Node.js 24.18, MySQL 8.4 on the same machine) and describe that setup only.

5.1 Tools and Technologies

Table 5.1: Tools and libraries (exact versions from package.json)

Layer	Tool	Version	Use
Runtime	Node.js; MySQL	24.18; 8.4 LTS	Server; storage (portable, port 3307)
Server	express; socket.io; mysql2	4.21.2; 4.8.1; 3.11.5	REST API; live hunts; pooled queries
Server	bcryptjs; jsonwebtoken; nodemailer	2.4.3; 9.0.2; 6.9.16	Hashes; tokens; email codes
Client	react, react-dom; react-router-dom; vite	18.3.1; 6.28.0; 5.4.11	Interface; routing; build
Client	three; @react-three/fiber; drei	0.169.0; 8.17.10; 9.117.3	3D models
Client	maplibre-gl; framer-motion; socket.io-client	4.7.1; 11.11.17; 4.8.1	Map on OpenFreeMap tiles; animation; hunts
Testing	node:test, fetch, socket.io-client	built in; 4.8.1	Unit, API and hunt tests
Data	Wikidata, OpenStreetMap, Wikimedia Commons, Poly Haven	CC-licensed or CC0	Coordinates, photos, textures

The project has about 2,300 lines of server code, 5,500 lines of client pages, components and libraries, 1,900 lines of style sheets, about 68,600 lines in the per-site 3D files, and eight JSON files of researched site data.

5.2 Module Implementation

`npm run db:init` creates the database, runs `schema.sql` and loads eight JSON files, one per region. Each record has coordinates, story, facts, hours, fees, Nepali text, photo credits, a 3D specification, ten questions and about six clues, giving 116 sites, 1,160 questions and 696 clues, plus a demo explorer and administrator. The algorithms of Chapter 4 are direct translations of Section 4.3. The core of `greedyInsert` in `tour.js`, abridged, is:

```
for (const i of left) for (let p = 0; p <= route.length; p++) {
  const s = simulate(insertAt(route, p, i), ...);
  if (!s) continue;                          // breaks a time window
  const ratio = sites[i].value / Math.max(1, s.finish - cur.finish);
  if (!best || ratio > best.ratio) best = { i, p, ratio };
}
route.splice(best.p, 0, best.i);       // insert the best, repeat
```

The quiz answer endpoint keeps the answer secret by shuffling the four options with a seeded generator of the run seed and question id, so the server can rebuild the order, and by remembering when each question was sent:

```
const sentAt = served.get(seed + ":" + q.id);
if (!sentAt) return res.status(409).json({ error: "Already answered." });
served.delete(key);                    // one answer per question per run
const elapsed = Date.now() - sentAt; // server clock, not the client's
if (elapsed > QUIZ_LIMIT_MS + QUIZ_GRACE_MS) choice = -1; // time-out
```

The hunt guess handler follows the same rule: the player is marked answered before any await, the time is the server's, and the reply carries points but no location.

5.3 Screens

The figures are screenshots of the running system at desktop width; further screens are in the appendix.



Figure 5.1: Home page with the hero model and the seven provinces

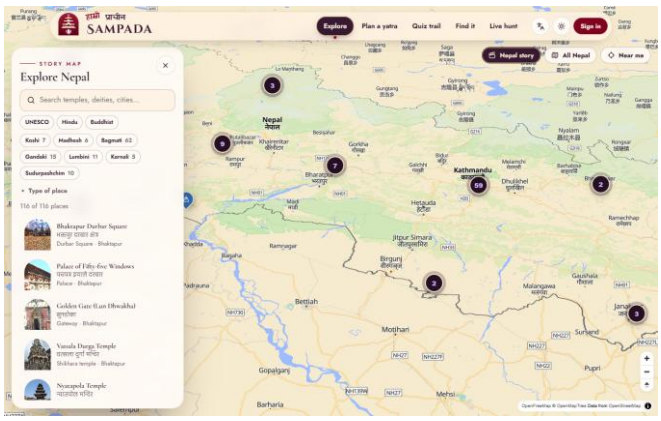


Figure 5.2: Explore: the Nepal map with province filters and the site list

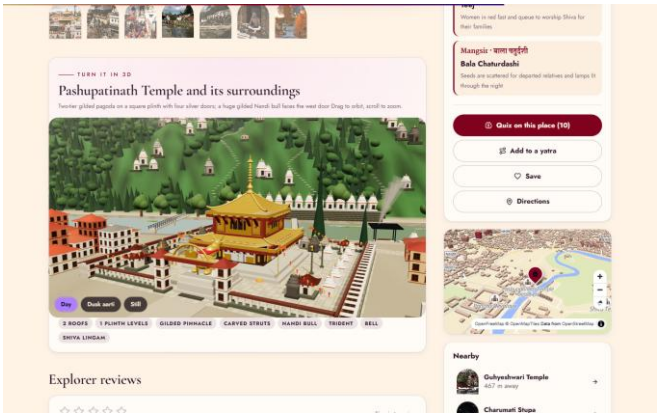


Figure 5.3: 3D model of a site in the dusk-aarti setting

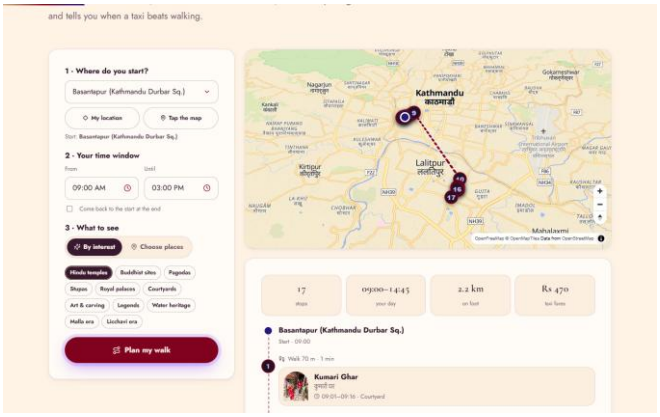


Figure 5.4: Yatra planner with the timed plan and its walk and taxi legs

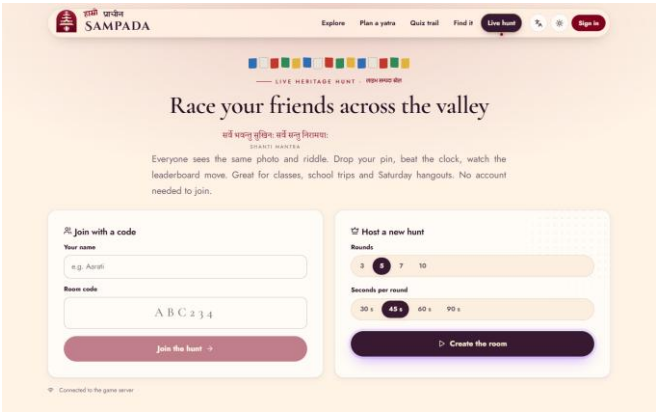


Figure 5.5: Live hunt with the room code and the leaderboard

5.4 Testing

Testing follows the risk. The algorithms are pure functions with rules that fail silently, so they get unit tests; the API holds validation, permissions and answer secrecy, so it gets end-to-end tests against a real database; the hunt is a time-driven multi-user protocol, so a script plays a whole hunt with three clients; and the interface is driven in a real browser.

5.4.1 Unit Tests

The twelve unit tests (`algorithms.test.js`, `npm test`) check each rule with known numbers. Recorded result: Pass: pass 12, fail 0.

Table 5.2: Unit test cases

ID	What is tested	Expected result	Result
UT-01	Haversine	Kathmandu to Patan Durbar Square 3.6–4.2 km	Pass
UT-02	Walk or taxi leg	About 140 m walked at no cost; Kathmandu to Bhaktapur is a taxi leg over Rs 500	Pass
UT-03	Planner hours	Every stop within site hours and the end time	Pass
UT-04	Closed site	A site open 15:00–16:00 in a 09:00–12:00 day is skipped with a reason	Pass
UT-05	Planner vs brute force	Same total value as the exact optimum on a 5-site instance	Pass
UT-06	Elo	50% for equal ratings; a win raises player and lowers question	Pass
UT-07	Adaptive picker	Closest to 70% success; due review first; null when all seen	Pass
UT-08	SM-2	Intervals 1, 6, then growing; a lapse resets; ease at least 1.3	Pass
UT-09	Find scoring	1,000 for an exact pin at time 0; 2 hints cost 200	Pass
UT-10	Hunt ranking	Points, then time; ties share a rank; same seed, same rounds	Pass
UT-11	Recommender	Interest match ranks first with its reason	Pass
UT-12	Scoring scope	5 km miss under 40 (Valley), over 600 (Nepal)	Pass

5.4.2 API Tests and Hunt Simulation

The eighteen API tests (`api-flow.test.js`, `npm run test:api`) run in sequence against the live server and a freshly seeded database. Recorded result: Pass: pass 18, fail 0.

Table 5.3: API test cases

ID	What is tested	Expected result	Result
AT-01	Health check	HTTP 200	Pass
AT-02	Catalog metadata	100+ sites, 7+ UNESCO zones, 500+ questions, all provinces; Karnali filter returns only Karnali	Pass
AT-03	Search and filters	“nyatapola” finds Nyatapola Temple; filters return only matches	Pass

ID	What is tested	Expected result	Result
AT-04	Monument detail	Nearby sites included; unknown slug 404	Pass
AT-05	Recommendations	Three or more cities, each with reasons	Pass
AT-06	Planner on real data	09:00–13:00 walk has 3+ stops and respects hours	Pass
AT-07	Planner bad input	No start, or end before start: 400	Pass
AT-08	Registration	Weak password, bad or duplicate email rejected; valid returns a token	Pass
AT-09	Login, protected routes	Wrong password 401; no token 401	Pass
AT-10	Adaptive quiz	No answer in the question; rating moves; a miss enters review	Pass
AT-11	GPS stamp	Far 422; on site accepted; first-stamp badge	Pass
AT-12	Favorites and reviews	One review per explorer; favorite toggles	Pass
AT-13	Walks	Yatri badge on save; share code opens without login; other user 403	Pass
AT-14	Find the Monument	No coordinates in rounds; replaying round 0 gives 409	Pass
AT-15	Explorer on admin route	HTTP 403	Pass
AT-16	Admin management	Latitude 40 and 2-option question refused; valid ones added; delete then 404	Pass
AT-17	Deactivation	Login 403; old token 401 at once	Pass
AT-18	Passport summary	Two stamps, 7 UNESCO zones, badge earned	Pass

`server/test/hunt-sim.mjs` opens three Socket.io clients (host, “Aarati”, “Bikash”) and plays a three-round hunt of 20 seconds. It checks that a duplicate name in another case is refused, that a non-host cannot start, that no round payload contains the monument id or coordinates, and that a second guess in a round is refused, then prints the ranking. Recorded result: Pass: ranking: 1.Host 189, 2.Aarati 189, 3.Bikash 5.

5.4.3 Browser Flow Tests

Unit and API tests cannot show that a page renders or a button works, so scripted flows drive the real pages in a browser at desktop width and at 375 px.

The flows were run by a script on one computer against the development server: 23 flows, 23 passed and 0 failed.

5.5 Results and Discussion

The system meets the objectives of Chapter 1: a bilingual catalog of 116 sites in all seven provinces (9 UNESCO zones); a 3D model for every site; a yatra planner; an adaptive quiz of 1,160 questions; two map games; live hunts for up to 30 players; and a passport with 11 badges. Everything works for guests except features that must remember a person.

5.5.1 Planner Quality and Speed

Speed was measured by calling `planTour` 20 times per size with real sites near Kathmandu Durbar Square, random values and a 09:00–17:00 day.

Table 5.4: Planner running time (measured, 20 runs)

Candidate sites	Median (ms)	Largest (ms)	Stops in plan
5	0.15	1.9	5
20	2.0	5.6	16
40	5.6	7.9	18
53 (every site within 25 km)	10.5	16.1	21

The largest realistic request takes about 10 ms, so the $O(n^4)$ bound is not a practical problem. For quality, the heuristic was compared with `bruteForce` on 40 random instances of seven sites within 8 km, a 09:00–14:00 window and random values. It found the optimal value in 27 of 40 (68 percent); in the other 13 the value was lower by 0.8 to 10.1 percent, 4.4 percent on average (1.4 percent over all 40). The plan is always valid, usually the best, and sometimes one modest stop short.

5.5.2 API Response Time and Client Size

A script sent 60 sequential requests to each endpoint on the same computer (the first includes warm-up).

Table 5.5: API response time (60 sequential requests)

Endpoint	First (ms)	Median (ms)	95th pct (ms)
GET /api/monuments (116 sites, 176 KB)	18	14.0	18.0
GET /api/play/quiz/next	16	14.8	16.1
POST /api/tours/plan (6 chosen sites)	16	15.1	16.2
POST /api/tours/plan (automatic, 53 candidates)	25	15.0	26.6

Medians are about 15 ms for every endpoint (eight measured, four shown). This floor comes from the measuring setup (sequential requests from a Node.js client on Windows), not the endpoints: the planner needs 0.2 to 10 ms and a catalog read a fraction of a millisecond. The honest conclusion is limited: on one computer no endpoint exceeded about 30 ms at the 95th percentile, and the table cannot rank endpoints. It says nothing about a real network, where the 176 KB catalog list matters more because the server installs no compression.

Table 5.6: Files fetched on the first visit (production build)

Chunk	Raw size	Gzip size
index (shell and shared code)	462 KB	121 KB
react, react-dom, react-router	165 KB	54 KB
framer-motion	122 KB	41 KB
three, react-three-fiber, drei	905 KB	249 KB
Style sheet of the shell	160 KB	29 KB
Total	1.8 MB	494 KB

Page chunks load on demand (Home 17 KB, Quiz 23 KB, Monument 68 KB, Hunt 102 KB, Admin 49 KB), and the map library (802 KB, 218 KB gzip) loads only on map pages. The build has 179 JavaScript files of 5.7 MB (1.8 MB gzip). A visitor fetches one site model at a time, plus eight group files (547 KB, 170 KB gzip) that load together on the first 3D view. The ten textures take 7.0 MB.

Keeping answers and time on the server holds up under the tests. The system has been run by its author and tested with scripts, not evaluated with real visitors or teachers, so no claim is made about learning benefit.

5.6 Limitations Found

- **One server process.** Hunts, running games, served quiz questions and the login throttle live in server memory; a restart ends all hunts, and two processes would not see each other’s rooms. A public service needs a shared store such as Redis and the Socket.io Redis adapter.
- **No general rate limit, compression or security headers.** Only login and email codes are limited. If `JWT_SECRET` is unset the server warns and uses a fixed development secret. The token is kept in local storage, so a cross-site scripting flaw would expose it; an `HttpOnly` cookie would be safer.

- **GPS stamps can be spoofed.** The server checks the coordinates are within about 60 m of the site (or the reported accuracy, up to 150 m) but cannot know they come from the sensor.
- **The travel model is an estimate.** Walking time is a straight line times 1.35 at a fixed speed, taxi time and fare are constants, and each site has one pair of opening hours with no festival closures. The heuristic matched the optimum in 27 of 40 small random cases and has no guarantee on larger ones.
- **Content is unreviewed.** The Nepali text comes from dictionaries and prepared text no professional translator has checked, and a new English sentence stays English until a dictionary entry is added. Stories, dates and answers come from listed public sources but no archaeologist or heritage official has reviewed them.
- **The first 3D view is heavy.** It fetches the 905 KB three.js chunk and the eight group model files (170 KB gzip) even though a site file replaces the group model for every site that has one.
- **Accessibility and coverage.** There was no test with screen reader users, and the map and 3D viewer were not checked for keyboard operation. The API tests need a running server and a seeded database, so they are not part of `npm test`; browser flows ran on one computer and one browser.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project set out to let a visitor explore the heritage of Nepal the way they already use their phone: by looking, playing and walking. The result, Hamro Prachin Sampada, is a working web system with a React client and a Node.js server. It covers 116 sites in all seven provinces with story, facts and photos in English and Nepali, a 3D model of every site, an interactive map, an audio guide, a passport with GPS stamps and badges, a yatra planner, an adaptive quiz, a photo-and-riddle map game and live multiplayer hunts.

The technical core is a set of algorithms written from scratch, tested and measured: a planner that solves a small orienteering problem with opening hours in about 10 ms for the largest request, with the optimal value in 27 of 40 random cases; an Elo quiz that targets about 70 percent success, with SM-2 reviews; an exponential-distance map game; a hunt engine that keeps time and answers on the server; and a recommender mixing interest, Bayesian rating, novelty and nearness.

The evidence of quality is 12 unit tests, 18 API tests, a three-player hunt simulation and scripted browser flows, with measured sizes and timings. The limits are in Section 5.6: a single server process, content not reviewed by curators or Nepali language experts, and no trial with real visitors. The system is a working, tested prototype and not yet a public service.

6.2 Lessons Learned

- **Keep rules in pure functions** so they are testable in milliseconds and comparable with a brute-force solver, which turned “the plan looks good” into “optimal in 68 percent of small cases”.
- **Decide who owns the clock and the answer.** Almost every way of cheating comes from trusting the client for time or answer; the rule is cheap at the start and costly later.
- **Measure honestly.** The latency table exposed a limit of the measuring setup, and the size table exposed a real problem, the group model files that load together.
- **Data is most of the work.** Collecting, checking and translating 116 sites, 1,160 questions and 696 clues took more effort than any algorithm.

6.3 Future Work

5. **Native review of Nepali** by a language teacher, with a button for users to propose better sentences.
6. **More sites, curator and teacher review.** Grow the catalog toward the full Department of Archaeology list, add seasonal and festival hours, have archaeologists check content, and run a study with a class to measure learning.
7. **Offline progressive web app** that caches catalog, photos and models so passport, audio guide, quiz and planner work on weak data.
8. **Augmented reality and a mobile app** for overlaying reconstructions of lost structures such as Kasthamandap, background GPS and festival notifications.
9. **Production hardening.** Move live state to a shared store, add rate limiting, compression and security headers, use an `HttpOnly` cookie, use a routing service for walking times, and run all tests automatically on every change.

REFERENCES

[1] UNESCO World Heritage Centre, “Kathmandu Valley,” World Heritage List, ref. 121. [Online]. Available: https://whc.unesco.org/en/list/121

[2] UNESCO World Heritage Centre, “Lumbini, the Birthplace of the Lord Buddha,” World Heritage List, ref. 666. [Online]. Available: https://whc.unesco.org/en/list/666

[3] “Govt begins restoring important heritage sites,” The Kathmandu Post, Oct. 6, 2016. [Online]. Available: https://kathmandupost.com/national/2016/10/06/govt-begins-restoring-important-heritage-sites

[4] C. Larman and V. R. Basili, “Iterative and incremental developments: A brief history,” Computer, vol. 36, no. 6, pp. 47–56, Jun. 2003, doi: 10.1109/MC.2003.1204375.

[5] E. F. Anderson, L. McLoughlin, F. Liarokapis, C. Peters, P. Petridis, and S. de Freitas, “Developing serious games for cultural heritage: A state-of-the-art review,” Virtual Reality, vol. 14, no. 4, pp. 255–275, 2010, doi: 10.1007/s10055-010-0177-3.

[6] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, “From game design elements to gamefulness: Defining ‘gamification’,” in Proc. 15th Int. Academic MindTrek Conf., Tampere, Finland, 2011, pp. 9–15, doi: 10.1145/2181037.2181040.

[7] J. Hamari, J. Koivisto, and H. Sarsa, “Does gamification work? A literature review of empirical studies on gamification,” in Proc. 47th Hawaii Int. Conf. System Sciences (HICSS), 2014, pp. 3025–3034, doi: 10.1109/HICSS.2014.377.

[8] M. Sailer and L. Homner, “The gamification of learning: A meta-analysis,” Educ. Psychol. Rev., vol. 32, no. 1, pp. 77–112, 2020, doi: 10.1007/s10648-019-09498-w.

[9] A. E. Elo, The Rating of Chessplayers, Past and Present. New York, NY, USA: Arco, 1978.

[10] S. Klinkenberg, M. Straatemeier, and H. L. J. van der Maas, “Computer adaptive practice of Maths ability using a new item response model for on the fly ability and difficulty estimation,” Comput. Educ., vol. 57, no. 2, pp. 1813–1824, 2011.

[11] R. Pelánek, “Applications of the Elo rating system in adaptive educational systems,” Comput. Educ., vol. 98, pp. 169–179, 2016, doi: 10.1016/j.compedu.2016.03.017.

[12] P. A. Wozniak and E. J. Gorzelanczyk, “Optimization of repetition spacing in the practice of learning,” Acta Neurobiol. Exp., vol. 54, pp. 59–62, 1994, doi: 10.55782/ane-1994-1003.

[13] H. L. Roediger III and J. D. Karpicke, “Test-enhanced learning: Taking memory tests improves long-term retention,” Psychol. Sci., vol. 17, no. 3, pp. 249–255, 2006, doi: 10.1111/j.1467-9280.2006.01693.x.

[14] B. Tabibian, U. Upadhyay, A. De, A. Zarezade, B. Schölkopf, and M. Gomez-Rodriguez, “Enhancing human learning via spaced repetition optimization,” Proc. Natl. Acad. Sci. U.S.A., vol. 116, no. 10, pp. 3988–3993, 2019, doi: 10.1073/pnas.1815156116.

[15] B. L. Golden, L. Levy, and R. Vohra, “The orienteering problem,” Naval Res. Logist., vol. 34, no. 3, pp. 307–318, Jun. 1987.

[16] P. Vansteenwegen, W. Souffriau, and D. Van Oudheusden, “The orienteering problem: A survey,” Eur. J. Oper. Res., vol. 209, no. 1, pp. 1–10, 2011, doi: 10.1016/j.ejor.2010.03.045.

[17] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou, “A survey on algorithmic approaches for solving tourist trip design problems,” J. Heuristics, vol. 20, no. 3, pp. 291–328, 2014, doi: 10.1007/s10732-014-9242-5.

[18] G. A. Croes, “A method for solving traveling-salesman problems,” Oper. Res., vol. 6, no. 6, pp. 791–812, 1958, doi: 10.1287/opre.6.6.791.

[19] R. W. Sinnott, “Virtues of the haversine,” Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[20] Khronos Group, “WebGL Specification, Version 1.0.” [Online]. Available: https://registry.khronos.org/webgl/specs/1.0/

[21] three.js authors, “three.js: JavaScript 3D library.” [Online]. Available: https://threejs.org

[22] Y. I. H. Parish and P. Müller, “Procedural modeling of cities,” in Proc. 28th Annu. Conf. Computer Graphics and Interactive Techniques (SIGGRAPH), 2001, pp. 301–308.

[23] P. Müller, P. Wonka, S. Haegler, A. Ulmer, and L. Van Gool, “Procedural modeling of buildings,” ACM Trans. Graph., vol. 25, no. 3, pp. 614–623, 2006, doi: 10.1145/1141911.1141931.

[24] I. Fette and A. Melnikov, “The WebSocket Protocol,” IETF, RFC 6455, Dec. 2011. [Online]. Available: https://www.rfc-editor.org/info/rfc6455

[25] Google, “Google Arts & Culture.” [Online]. Available: https://artsandculture.google.com

[26] “List of monuments in Nepal,” Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/List_of_monuments_in_Nepal

[27] D. Vrandečić and M. Krötzsch, “Wikidata: A free collaborative knowledgebase,” Commun. ACM, vol. 57, no. 10, pp. 78–85, 2014, doi: 10.1145/2629489.

[28] Sketchfab, “Kathmandu – 3D views before and after the earthquake,” Sketchfab Community Blog. [Online]. Available: https://sketchfab.com/blogs/community/kathmandu-3d-views-before-and-after-the-earthquake/

[29] M. Haklay and P. Weber, “OpenStreetMap: User-generated street maps,” IEEE Pervasive Comput., vol. 7, no. 4, pp. 12–18, Oct.–Dec. 2008, doi: 10.1109/MPRV.2008.80.

[30] P. A. Woźniak, "Optimization of learning," M.S. thesis, Univ. of Technology, Poznań, Poland, 1990 (section 3.2, the SM-2 algorithm).

[31] J. Carbonell and J. Goldstein, "The use of MMR, diversity-based reranking for reordering documents and producing summaries," in Proc. 21st Annu. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval, Melbourne, Australia, 1998, pp. 335–336, doi: 10.1145/290941.291025.

[32] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015.

[33] N. Provos and D. Mazières, "A future-adaptable password scheme," in Proc. USENIX Annu. Tech. Conf., Monterey, CA, USA, 1999, pp. 81–92.

APPENDIX A: RUNNING THE SYSTEM

1. Install Node.js 20 or later. Extract MySQL Community Server 8.4 into `.mysql` and run `setup-mysql.cmd` once, then `start-mysql.cmd` (port 3307).
2. In `server`, copy `.env.example` to `.env` and set `JWT_SECRET`; optionally set `SMTP_USER` and `SMTP_PASS` for Gmail.
3. Run `npm install` and `npm run db:init` in `server` to create and seed the database (116 sites, 1,160 questions, 696 clues).
4. Start the API with `npm run dev` in `server` (port 5050) and the client with `npm install` then `npm run dev` in `client` (port 5174).
5. Sign in with the demo explorer or administrator account listed in the README.

The complete source code is available in the public repository https://github.com/itsniranjannn/Prachin-Sampada.